



Digital Design

Chapter 3: Sequential Logic Design -- Controllers

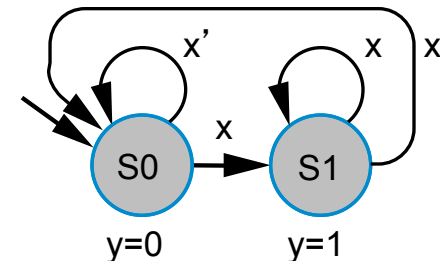
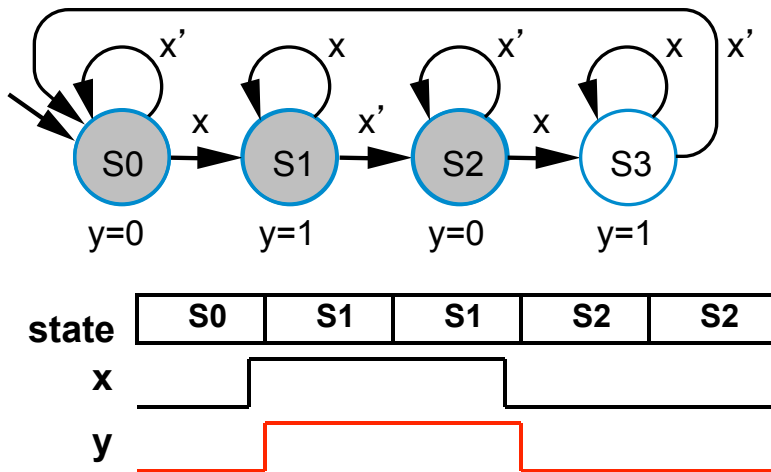
Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
<http://www.ddvahid.com>

Copyright © 2007 Frank Vahid

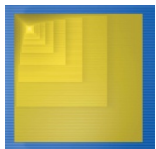
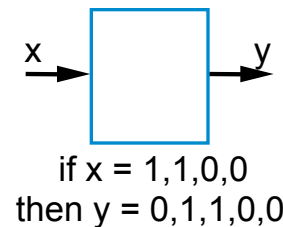
Instructors of courses requiring Vahid's *Digital Design* textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may not be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see <http://www.ddvahid.com> for information.

State Reduction (State Minimization)

- **Goal:** Reduce number of states in FSM *without* changing behavior
 - Fewer states potentially reduces size of state register
- Consider the two FSMs below with $x=1$, then 1, then 0, 0
Inputs: x ; Outputs: y



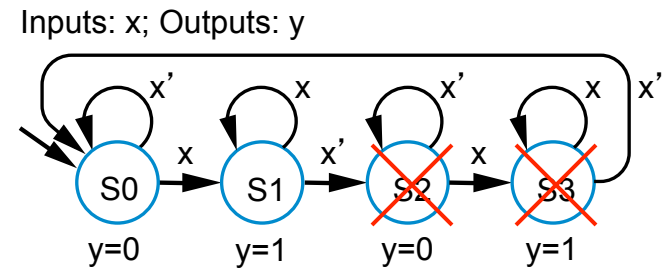
*For the same sequence of inputs,
the output of the two FSMs is the same*



State Reduction: Equivalent States

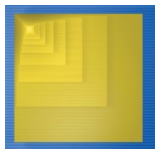
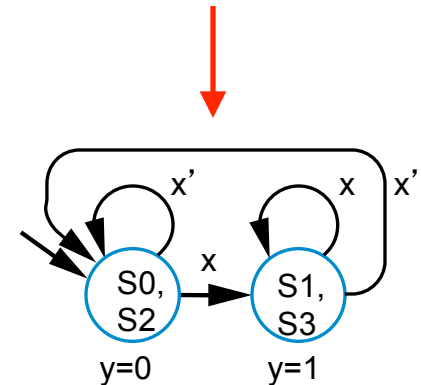
Two states are equivalent if:

1. They assign the same values to outputs
 - e.g. **S0** and **S2** both assign y to 0,
 - **S1** and **S3** both assign y to 1
2. AND, **for all possible sequences of inputs**, the FSM outputs will be the same starting from either state
 - e.g. say $x=1,1,0,0,\dots$
 - starting from **S1**, $y=1,1,0,0,\dots$
 - starting from **S3**, $y=1,1,0,0,\dots$



States S0 and S2 equivalent

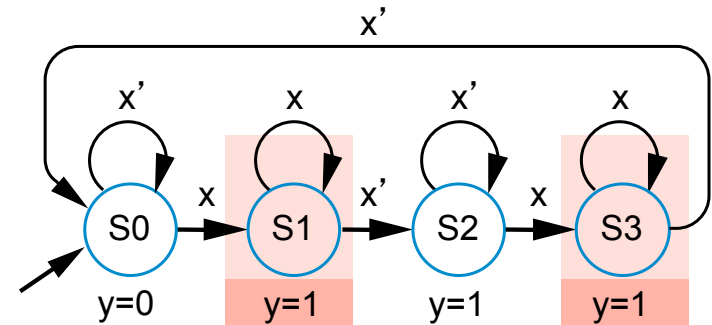
States S1 and S3 equivalent



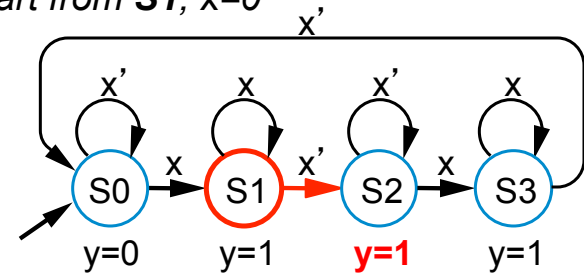
State Reduction: Example with no Equivalencies

- Another example...
- State **S0** is not equivalent with any other state since its output ($y=0$) differs from other states' output
- Consider state **S1** and **S3**
 - Outputs are initially the same ($y=1$)
 - From **S1**, when $x=0$, go to **S2** where $y=1$
 - From **S3**, when $x=0$, go to **S0** where $y=0$
 - Outputs differ, so **S1** and **S3** are *not* equivalent.

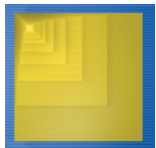
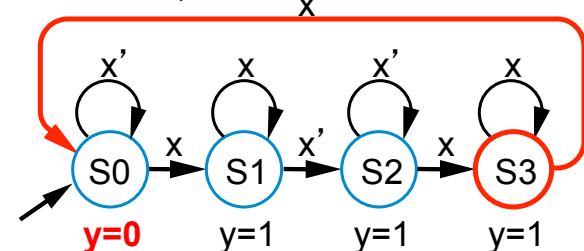
Inputs: x ; Outputs: y



Start from **S1**, $x=0$

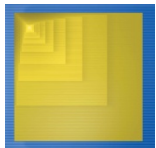


Start from **S3**, $x=0$



Nonequivalent States

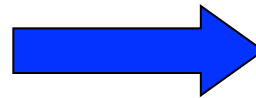
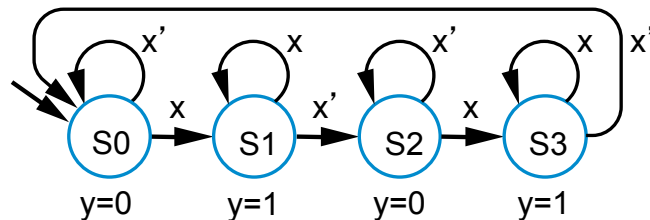
- If two state's outputs are not equivalent, the two states are not equivalent.
- If two states' next states are not equivalent for a given input value, then the two states are also not equivalent.
- Using these concepts of nonequivalent states, we will describe an algorithm for reducing an FSM's number of states.



State Reduction with Implication Tables

- State reduction through visual inspection (what we did in the last few slides) isn't reliable and cannot be automated – a more methodical approach is needed: **implication tables**
- Example:

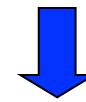
Inputs: x ; Outputs: y



S0				
S1				
S2				
S3				
	S0	S1	S2	S3

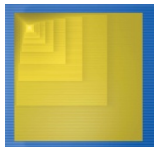
Redundant

Diagonal



S1			
S2			
S3			
	S0	S1	S2

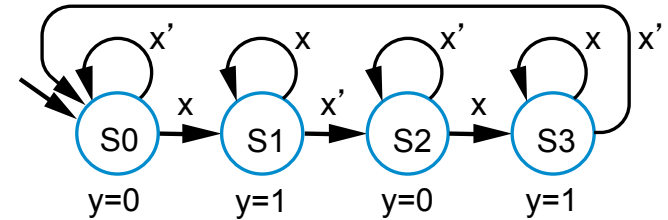
- To compare every pair of states, construct a table of *state pairs* (above right)
- Remove redundant state pairs, and state pairs along the diagonal since a state is equivalent to itself (right)



State Reduction with Implication Tables

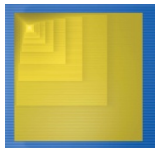
- Mark (with an X) state pairs with different outputs as non-equivalent:
 - (**S1**, **S0**): At **S1**, $y=1$ and at **S0**, $y=0$. So **S1** and **S0** are non-equivalent.
 - (**S2**, **S0**): At **S2**, $y=0$ and at **S0**, $y=0$. So we don't mark **S2** and **S0** now.
 - (**S2**, **S1**): Non-equivalent
 - (**S3**, **S0**): Non-equivalent
 - (**S3**, **S1**): Don't mark
 - (**S3**, **S2**): Non-equivalent
- We can see that **S2** & **S0** might be equivalent and **S3** & **S1** might be equivalent, but only if their next states are equivalent (remember the example from two slides ago)

Inputs: x ; Outputs: y



S1			
S2			
S3			
	S0	S1	S2

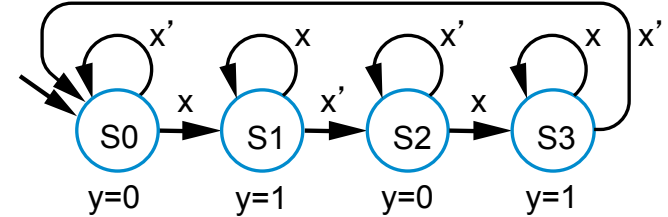
a



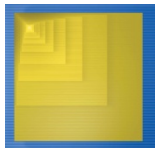
State Reduction with Implication Tables

- We need to check each unmarked state pair's next states
- We can start by listing what each unmarked state pair's next states are for every combination of inputs
 - (**S2, S0**)
 - From **S2**, when $x=1$ go to **S3**
From **S0**, when $x=1$ go to **S1**
So we add (**S3, S1**) as a next state pair
 - From **S2**, when $x=0$ go to **S2**
From **S0**, when $x=0$ go to **S0**
So we add (**S2, S0**) as a next state pair
 - (**S3, S1**)
 - By a similar process, we add the next state pairs (**S3, S1**) and (**S0, S2**)

Inputs: x ; Outputs: y

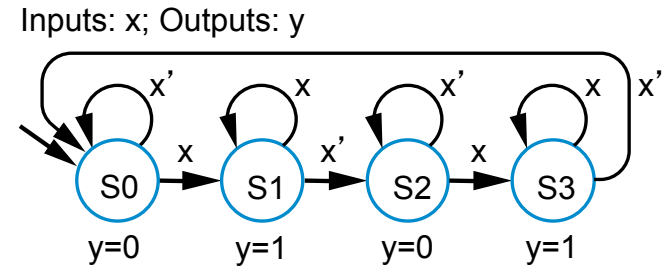


S1			
S2	(S3, S1) (S2, S0)		
S3		(S3, S1) (S0, S2)	
	S0	S1	S2

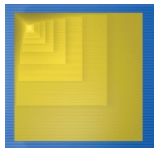


State Reduction with Implication Tables

- Next we check every unmarked state pair's next state pairs
- We mark the state pair if one of its next state pairs is marked
 - **(S2, S0)**
 - Next state pair **(S3, S1)** is not marked
 - Next state pair **(S2, S0)** is not marked
 - So we do nothing and move on
 - **(S3, S1)**
 - Next state pair **(S3, S1)** is not marked
 - Next state pair **(S0, S2)** is not marked
 - So we do nothing and move on



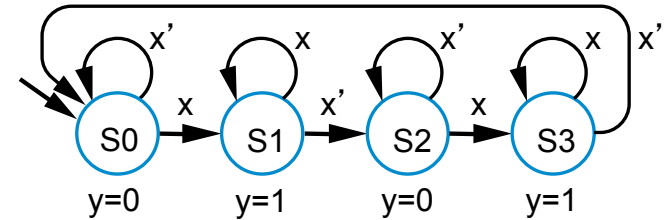
S1			
S2	(S3, S1) (S2, S0)		
S3		(S3, S1) (S0, S2)	
	S0	S1	S2



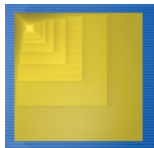
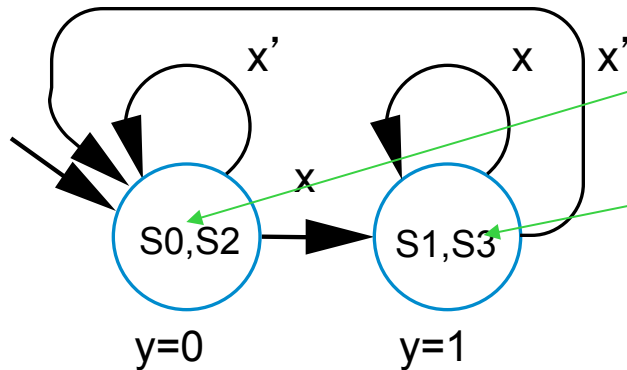
State Reduction with Implication Tables

- We just made a *pass* through the implication table
 - Make additional passes until no change occurs
- Then merge the unmarked state pairs – they are equivalent

Inputs: x ; Outputs: y

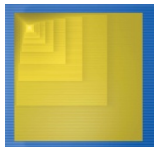


S1			
S2	(S3, S1) (S2, S0)		
S3		(S3, S1) (S0, S2)	
	S0	S1	S2



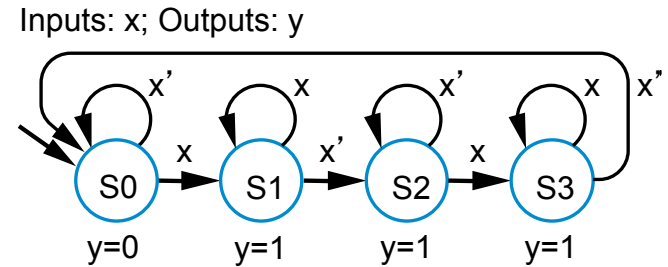
State Reduction with Implication Tables

Step	Description
1 <i>Mark state pairs having different outputs as nonequivalent</i>	States having different outputs obviously cannot be equivalent.
2 <i>For each unmarked state pair, write the next state pairs for the same input values</i>	
3 <i>For each unmarked state pair, mark state pairs having nonequivalent next-state pairs as nonequivalent. Repeat this step until no change occurs, or until all states are marked.</i>	States with nonequivalent next states for the same input values can't be equivalent. Each time through this step is called a <i>pass</i> .
4 <i>Merge remaining state pairs</i>	Remaining state pairs must be equivalent.

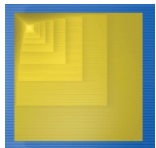


State Reduction Example

- Given FSM on the right
 - Step 1:** Mark state pairs having different outputs as nonequivalent

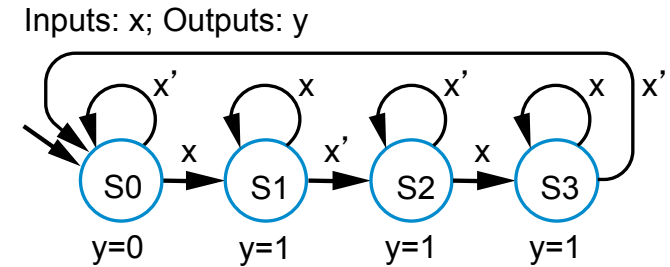


S1			
S2			
S3			
	S0	S1	S2



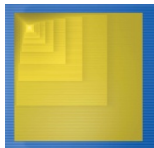
State Reduction Example

- Given FSM on the right
 - Step 1:** Mark state pairs having different outputs as nonequivalent
 - Step 2:** For each unmarked state pair, write the next state pairs for the same input values



S1			
S2		(S2, S2) (S3, S1)	
S3		(S0, S2) (S3, S1)	(S0, S2) (S3, S3)
	S0	S1	S2

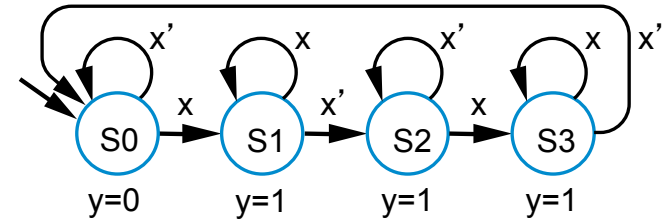
a



State Reduction Example

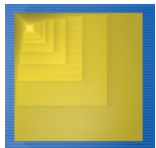
- Given FSM on the right
 - Step 1:** Mark state pairs having different outputs as nonequivalent
 - Step 2:** For each unmarked state pair, write the next state pairs for the same input values
 - Step 3:** For each unmarked state pair, mark state pairs having nonequivalent next state pairs as nonequivalent.
 - Repeat this step until no change occurs, or until all states are marked.
 - Step 4:** Merge remaining state pairs

Inputs: x ; Outputs: y



S1			
S2		(S2, S2) (S3, S1)	
S3		(S0, S2) (S3, S1)	(S0, S2) (S3, S3)
	S0	S1	S2

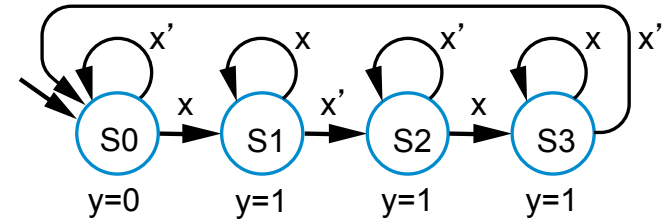
a



State Reduction Example

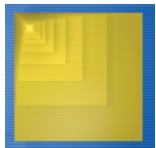
- Given FSM on the right
 - Step 1:** Mark state pairs having different outputs as nonequivalent
 - Step 2:** For each unmarked state pair, write the next state pairs for the same input values
 - Step 3:** For each unmarked state pair, mark state pairs having nonequivalent next state pairs as nonequivalent.
 - Repeat this step until no change occurs, or until all states are marked.
 - Step 4:** Merge remaining state pairs

Inputs: x ; Outputs: y

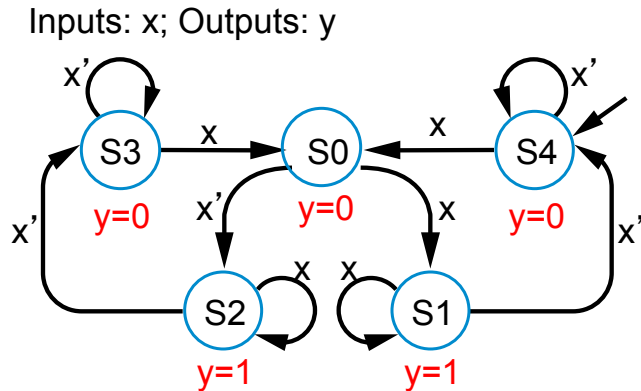


S1			
S2		(S2, S2) (S3, S1)	
S3		(S0, S2) (S3, S1)	(S0, S2) (S3, S3)
	S0	S1	S2

*All state pairs are marked –
there are no equivalent
state pairs to merge*



A Larger State Reduction Example



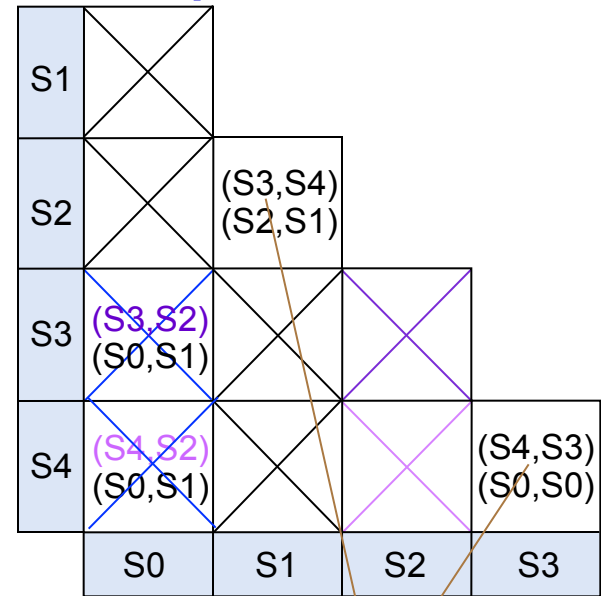
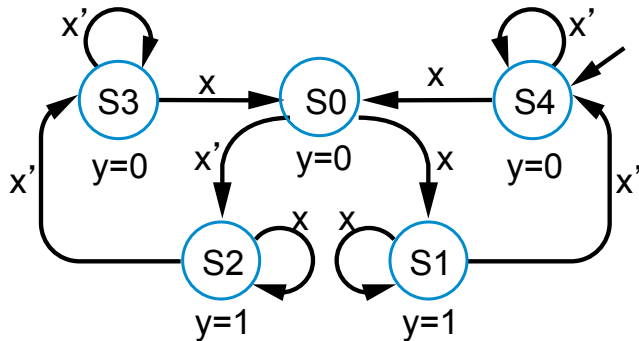
S1				
S2				
S3				
S4				
	S0	S1	S2	S3

- **Step 1:** Mark state pairs having different outputs as nonequivalent
- **Step 2:** For each unmarked state pair, write the next state pairs for the same input values
- **Step 3:** For each unmarked state pair, mark state pairs having nonequivalent next state pairs as nonequivalent.
 - Repeat this step until no change occurs, or until all states are marked.
- **Step 4:** Merge remaining state pairs



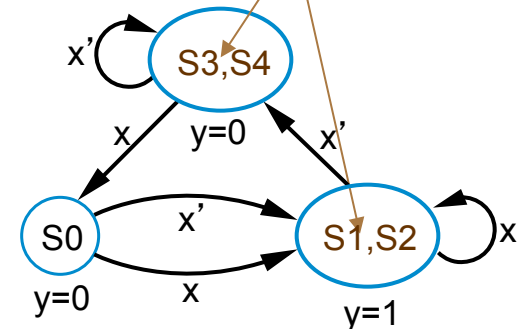
A Larger State Reduction Example

Inputs: x ; Outputs: y



- **Step 1:** Mark state pairs having different outputs as nonequivalent
- **Step 2:** For each unmarked state pair, write the next state pairs for the same input values
- **Step 3:** For each unmarked state pair, mark state pairs having **nonequivalent** next state pairs as nonequivalent.
 - Repeat this step until no change occurs, or until all states are marked.
- **Step 4:** Merge remaining state pairs

Inputs: x ; Outputs: y

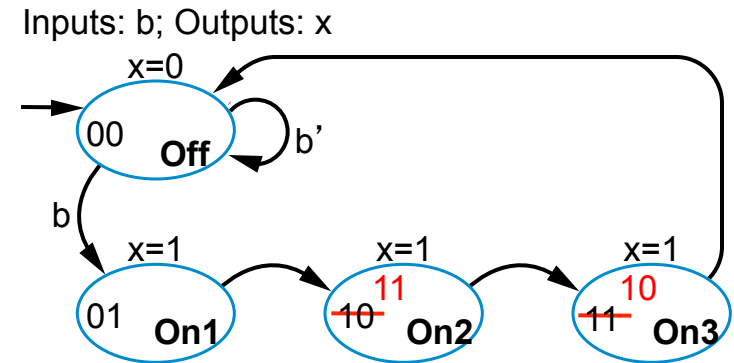


- Inputs: x ; Outputs: z
-
- The diagram shows a 16-state sequential circuit with states SA through SN. Each state has an output z (0 or 1). Transitions are labeled with inputs x and x' . The circuit includes several self-loops and cycles, such as a cycle involving states SA, SB, SH, SI, SO, SM, SN, SL, SK, SG, SD, SE, and SF.



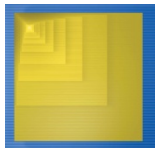
State Encoding

- **Encoding:** Assigning a unique bit representation to each state
- Different encodings may optimize size, or tradeoff size and performance
- **Minimum bit-width Binary Encoding:** Using the fewest number of bits
- Consider 3-Cycle Laser Timer...
 - Example 3.7's encoding: **15** gate inputs
 - Try **alternative encoding**
 - $x = s1 + s0$
 - $n1 = s0$
 - $n0 = s1' b + s1' s0$
 - Only **8** gate inputs, reduces size without increasing delay



	Inputs			Outputs			
	s1	s0	b	x	n1	n0	
<i>Off</i>	0	0	0	0	0	0	
	0	0	1	0	0	1	
<i>On1</i>	0	1	0	1	1	0 1	
	0	1	1	1	1	0	
<i>On2</i>	1	0 1	0	1	1	1 0	
	1	0	1	1	1	1	
<i>On3</i>	1	1 0	0	1	0	0	
	1	1	1	1	0	0	

4!=24 possibilities on 4 states with 2 bits.



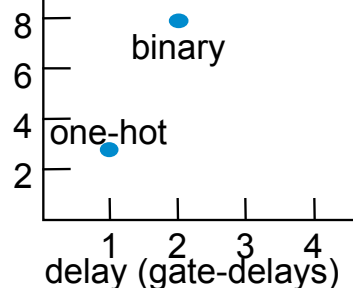
State Encoding: One-Hot Encoding

- **One-hot encoding**

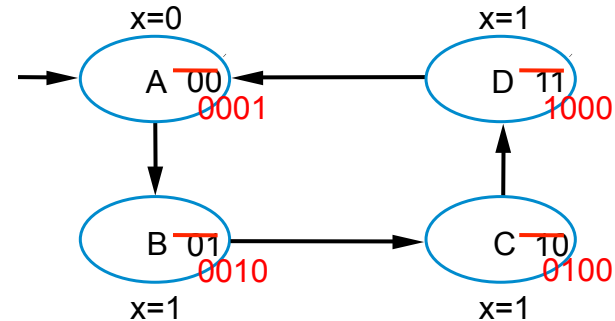
- One bit per state – a bit being '1' corresponds to a particular state, increase speed
- Alternative to *minimum bit-width encoding* in previous example
- For A, B, C, D: A: 0001, B: 0010, C: 0100, D: 1000

- Example: FSM that outputs 0, 1, 1, 1

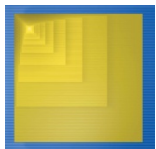
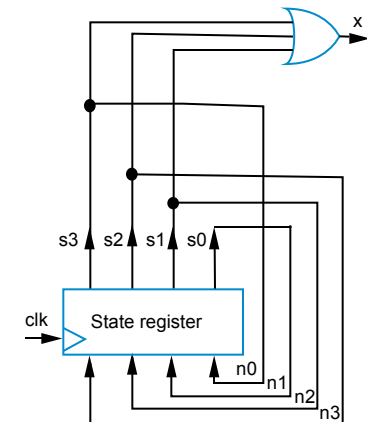
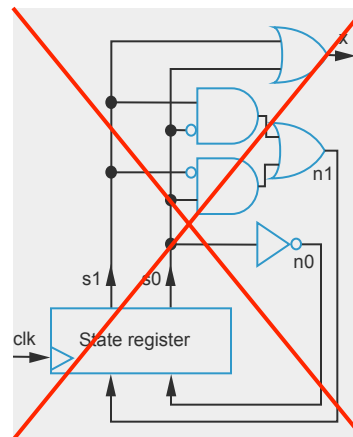
- Equations if one-hot encoding:
 - $n3 = s2$; $n2 = s1$; $n1 = s0$;
 - $x = s3 + s2 + s1$
- Fewer gates and only one level of logic – less delay than two levels, so faster clock frequency



Inputs: none; Outputs: x

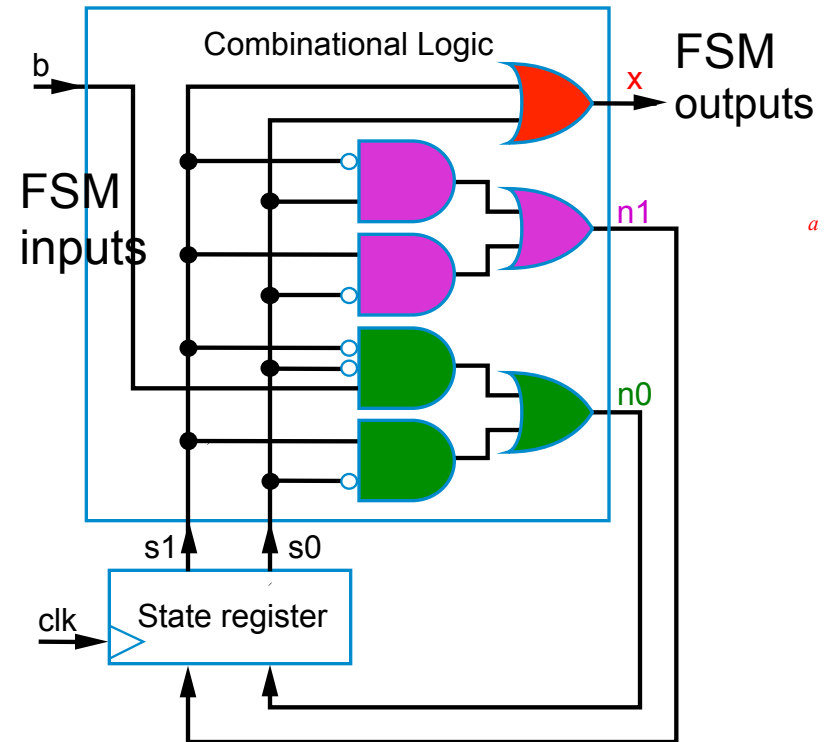

~~| | Inputs | | Outputs | | | |
|---|--------|----|---------|----|---|--|
| | s1 | s0 | n1 | n0 | x | |
| A | 0 | 0 | 0 | 1 | 0 | |
| B | 0 | 1 | 1 | 0 | 1 | |
| C | 1 | 0 | 1 | 1 | 1 | |
| D | 1 | 1 | 0 | 0 | 1 | |~~

	Inputs				Outputs				
	s3	s2	s1	s0	n3	n2	n1	n0	x
A	0	0	0	1	0	0	1	0	0
B	0	0	1	0	0	1	0	0	1
C	0	1	0	0	1	0	0	0	1
D	1	0	0	0	0	0	0	1	1



3 CycleLaser Timer Example (cont)

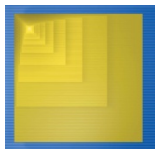
	Inputs			Outputs		
	s1	s0	b	x	n1	n0
<i>Off</i>	0	0	0	0	0	0
	0	0	1	0	0	1
<i>On1</i>	0	1	0	1	1	0
	0	1	1	1	1	0
<i>On2</i>	1	0	0	1	1	1
	1	0	1	1	1	1
<i>On3</i>	1	1	0	1	0	0
	1	1	1	1	0	0



$$x = s1 + s0$$

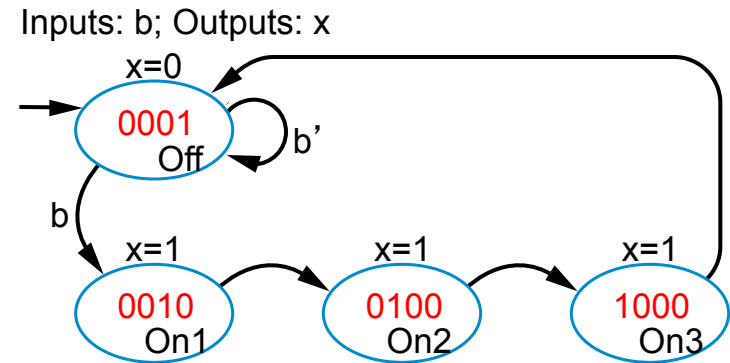
$$n1 = s1' s0 + s1 s0'$$

$$n0 = s1' s0' b + s1 s0'$$



One-Hot Encoding Example: Three-Cycles-High Laser Timer

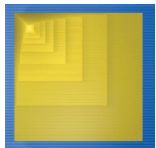
- Four states – Use four-bit one-hot encoding
 - State table leads to equations:
 - $x = s_3 + s_2 + s_1$
 - $n_3 = s_2$
 - $n_2 = s_1$
 - $n_1 = s_0 \cdot b$
 - $n_0 = s_0 \cdot b' + s_3$
 - Smaller
 - $3+0+0+2+(2+2) = \mathbf{9}$ gate inputs
 - Earlier binary encoding (Ch 3): **15** gate inputs
 - Faster
 - Critical path: $n_0 = s_0 \cdot b' + s_3$
 - Previously: $n_0 = s_1' s_0' b + s_1 s_0'$



	Inputs					Outputs				
	s3	s2	s1	s0	b	x	n3	n2	n1	n0
Off	0	0	0	1	0	0	0	0	0	1
	0	0	0	1	1	0	0	0	1	0
On1	0	0	1	0	0	1	0	1	0	0
	0	0	1	0	1	1	0	1	0	0
On2	0	1	0	0	0	1	1	0	0	0
	0	1	0	0	1	1	1	0	0	0
On3	1	0	0	0	0	1	0	0	0	1
	1	0	0	0	1	1	0	0	0	1

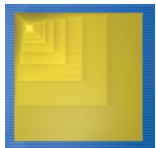
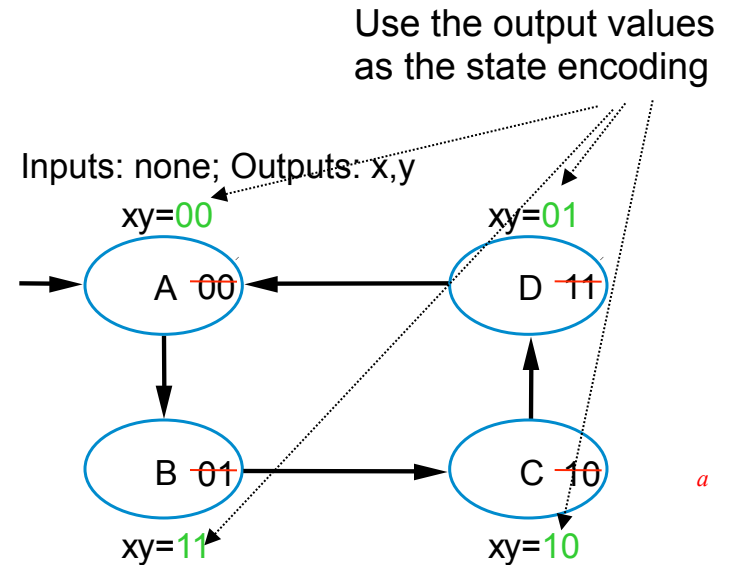
One-Hot Encoding

- What if we had 1000 states?
- Requires 10 bit state register with binary but 1000 with one-hot



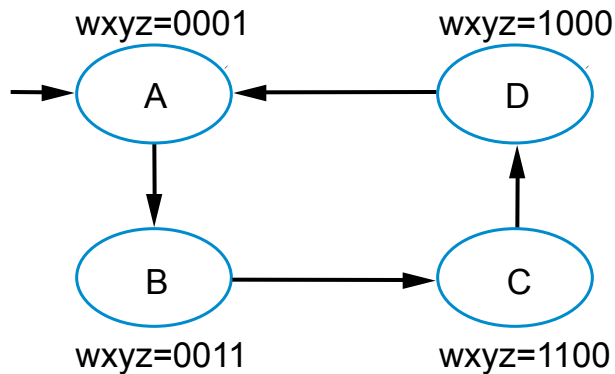
Output Encoding

- **Output encoding:** Encoding method where the state encoding is same as the **output values**
 - Possible if enough outputs (at least as many outputs as it has bits in a binary encoding, at least 2 outputs x and y for 2 bits), all states with unique output values
 - No need to use logic to generate outputs



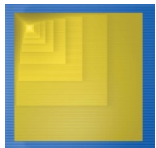
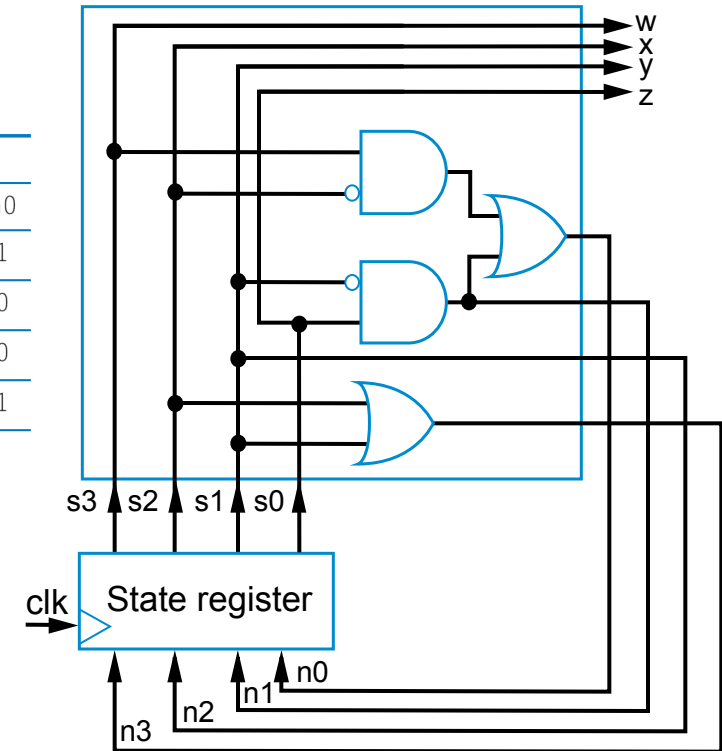
Output Encoding Example: Sequence Generator

Inputs: none; Outputs: w, x, y, z



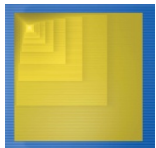
	Inputs				Outputs			
	s3	s2	s1	s0	n3	n2	n1	n0
A	0	0	0	1	0	0	1	1
B	0	0	1	1	1	1	0	0
C	1	1	0	0	1	0	0	0
D	1	0	0	0	0	0	0	1

- Generate sequence 0001, 0011, 1110, 1000, repeat
 - FSM shown
- Use output values as state encoding
- Create state table
- Derive equations for next state
 - $n3 = s1 + s2$; $n2 = s1$; $n1 = s1' s0$; $n0 = s1' s0 + s3 s2$
 - Has more logic than earlier.

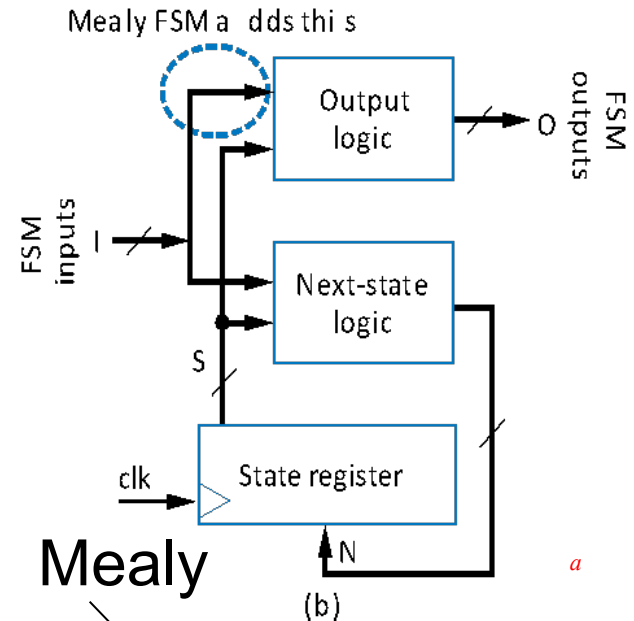
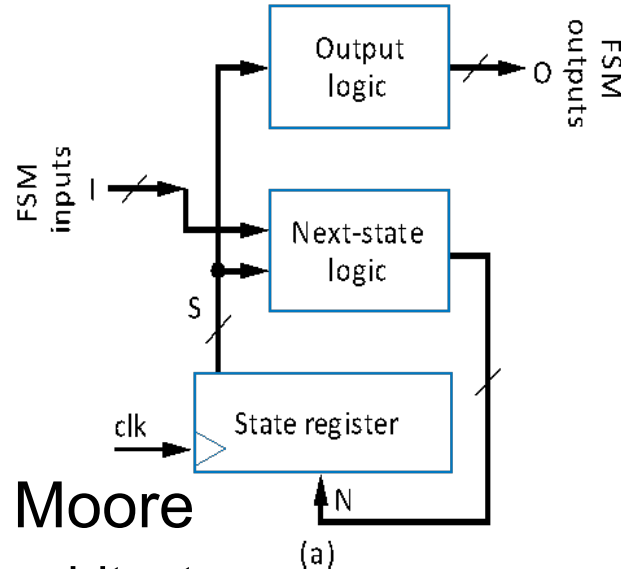
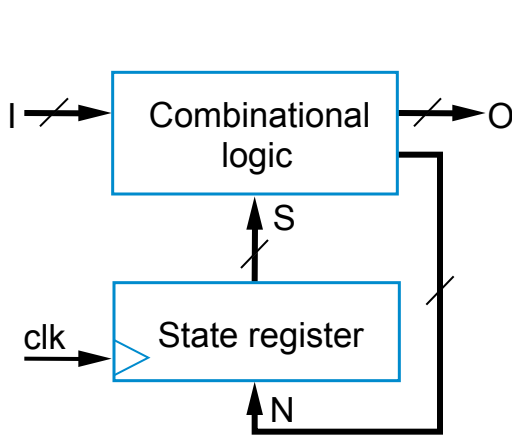


Encoding

- Binary, One-hot, Output encodings may result in smaller, faster circuitry depending on the example.
- Automated tools try a combination of approaches.



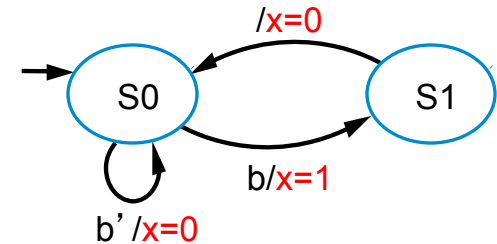
Moore vs. Mealy FSMs



Mealy FSM adds this

Mealy

Inputs: b; Outputs: x



Graphically: show **outputs** with arcs, not with states

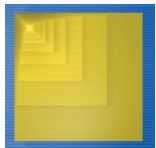
FSM implementation architecture

- State register and logic
- More detailed view

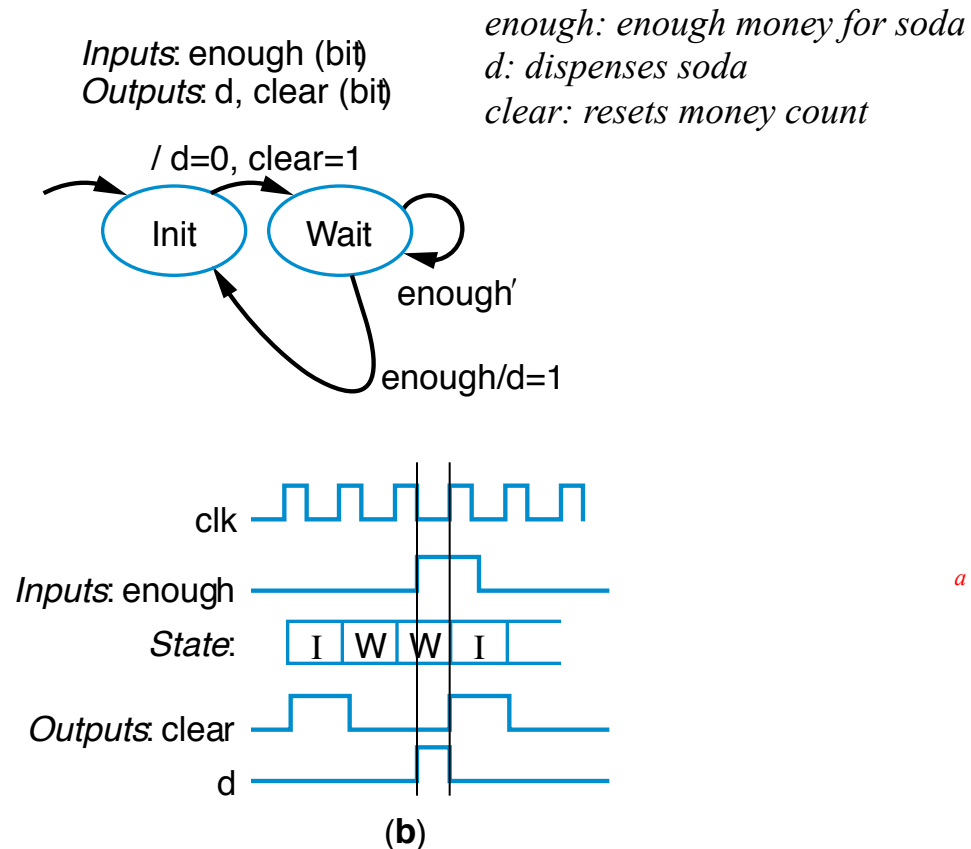
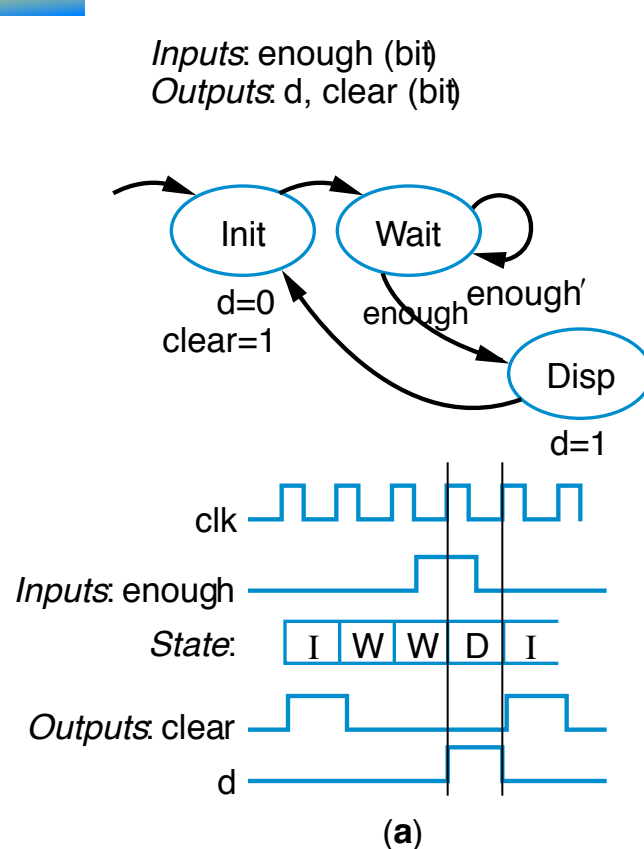
- Next state logic – function of present state and FSM inputs
- Output logic

- If function of present state only – **Moore FSM**

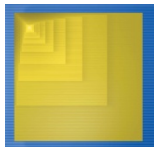
- If function of present state and FSM inputs – **Mealy FSM**



Mealy FSMs May Have Fewer States

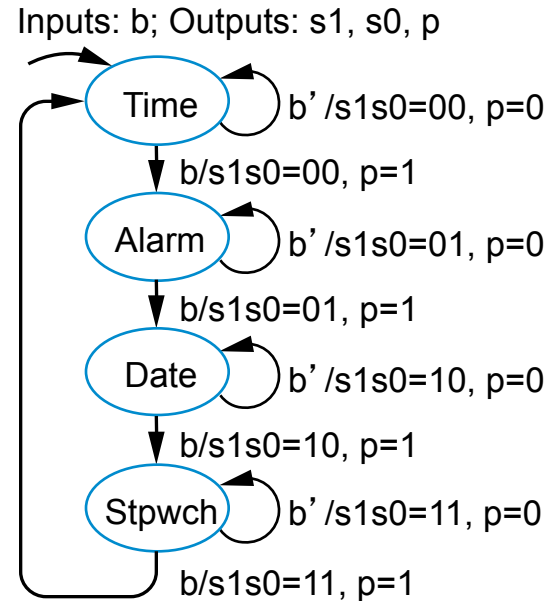


- Soda dispenser example: Initialize, wait until enough, dispense
 - Moore: 3 states; Mealy: 2 states

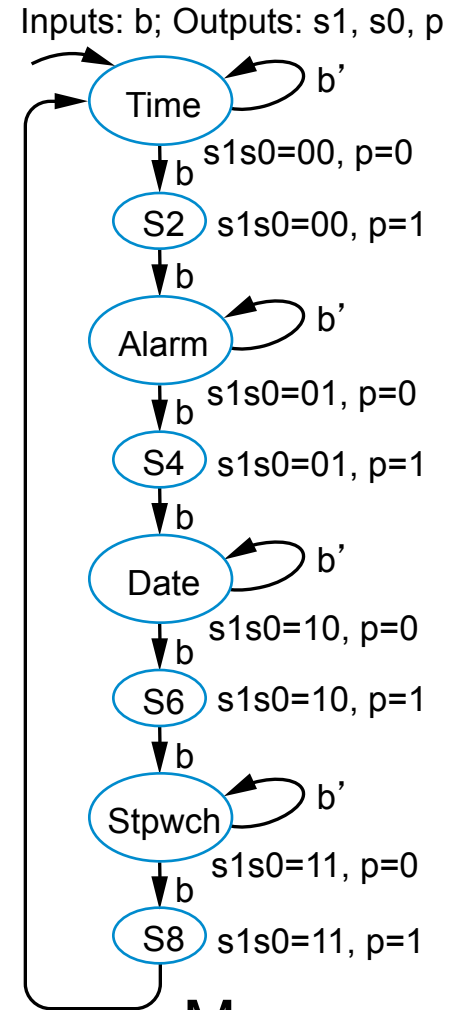


Mealy vs. Moore

- Q: Which is Moore, and which is Mealy?
- A: Mealy on left, Moore on right
 - Mealy outputs on arcs, meaning outputs are function of state AND INPUTS
 - Moore outputs in states, meaning outputs are function of state only



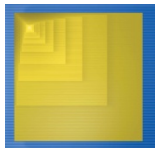
Mealy



Moore

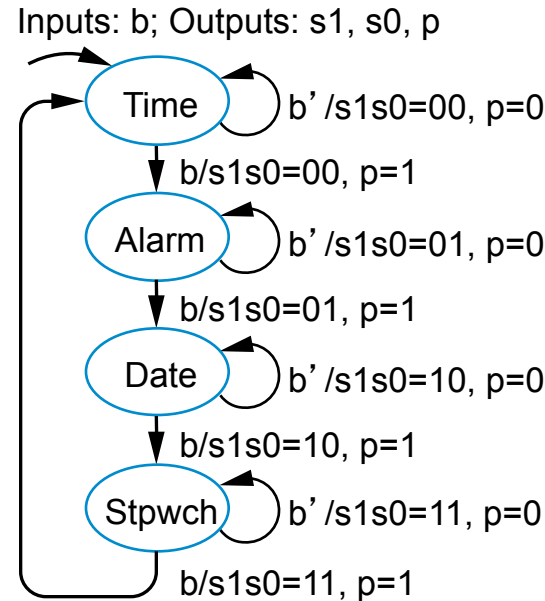
Example is wristwatch: pressing button b changes display (s1s0) and also causes beep (p=1)

Assumes button press is synchronized to occur for one cycle only

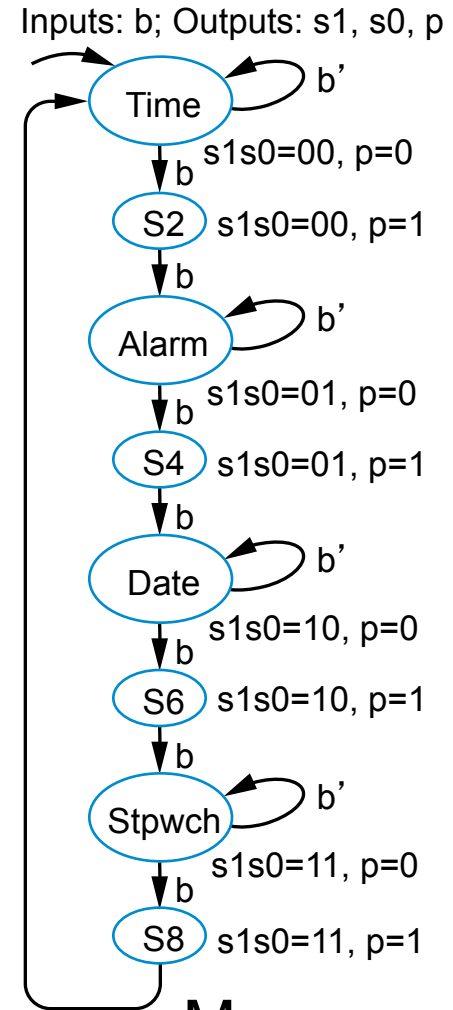


Mealy vs. Moore Example: Beeping Wristwatch

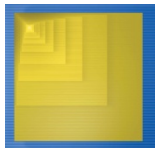
- Button b
 - Sequences mux select lines $s1s0$ through 00, 01, 10, and 11
 - Each value displays different internal register
 - Each unique button press should cause 1-cycle beep, with $p=1$ being beep
- Must wait for button to be released (b') and pushed again (b) before sequencing
 - Note that Moore requires unique state to pulse p , while Mealy pulses p on arc
- Tradeoff: Mealy's pulse on p may not last one full cycle



Mealy



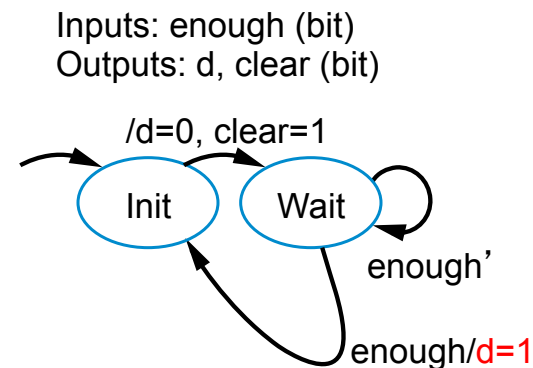
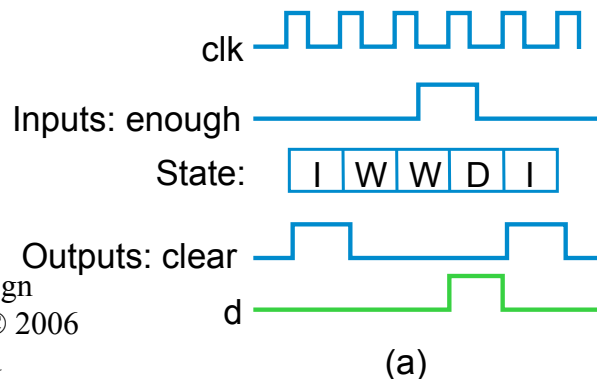
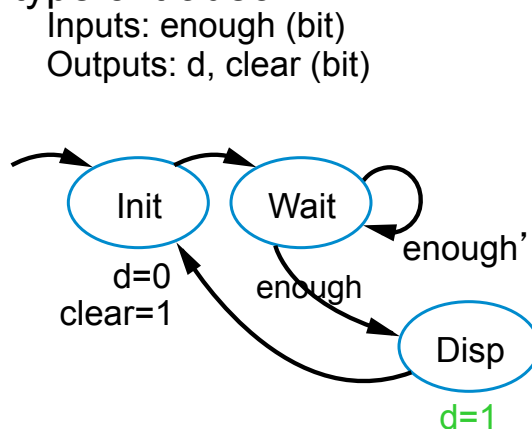
Moore



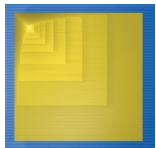
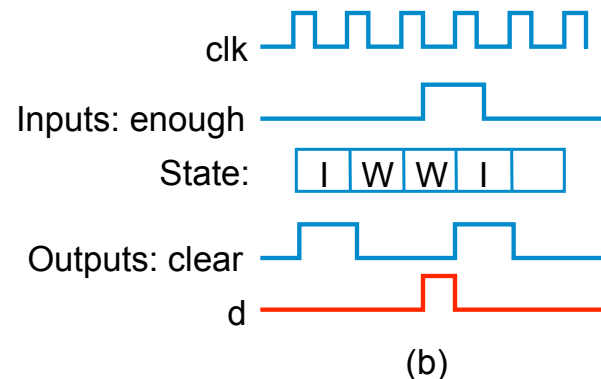
Mealy vs. Moore Tradeoff

- Mealy may have fewer states, but drawback is that its outputs change mid-cycle if input changes (**Mealy output not synchronized with clock edge, state transitions are synchronized**)
 - Note earlier soda dispenser example
 - Mealy had fewer states, but output **d not 1 for full cycle**
 - Represents a type of tradeoff

Moore

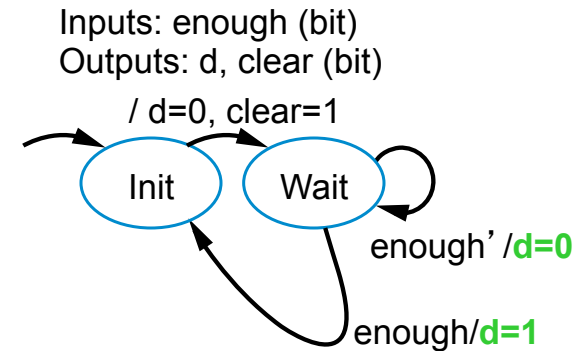


Mealy

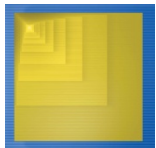


Implementing a Mealy FSM

- Straightforward
 - Convert to state table
 - Derive equations for each output
 - Key difference from Moore: External outputs (d , $clear$) may have **different value** in same state, depending on input values



	Inputs		Outputs		
	s0	enough	n0	d	clear
<i>Init</i>	0	0	1	0	1
	0	1	1	0	1
<i>Wait</i>	1	0	1	0	0
	1	1	0	1	0



Mealy vs. Moore

- Mealy fewer states
- Mealy timing issues

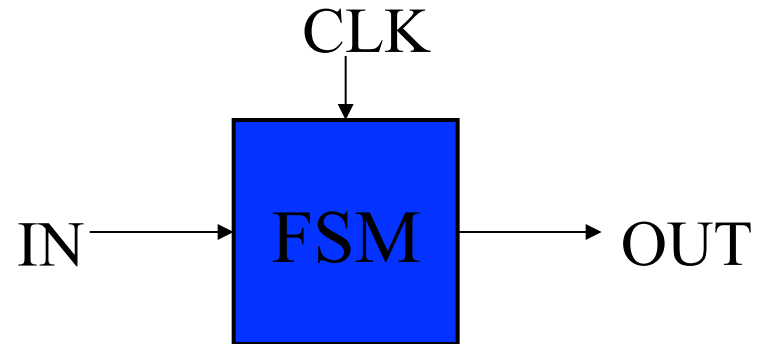


Finite State Machines

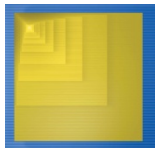
- Example: Positive Edge Detector**

Bits are received one at a time (one per cycle),
such as: 000111010 $\longrightarrow$ *time*

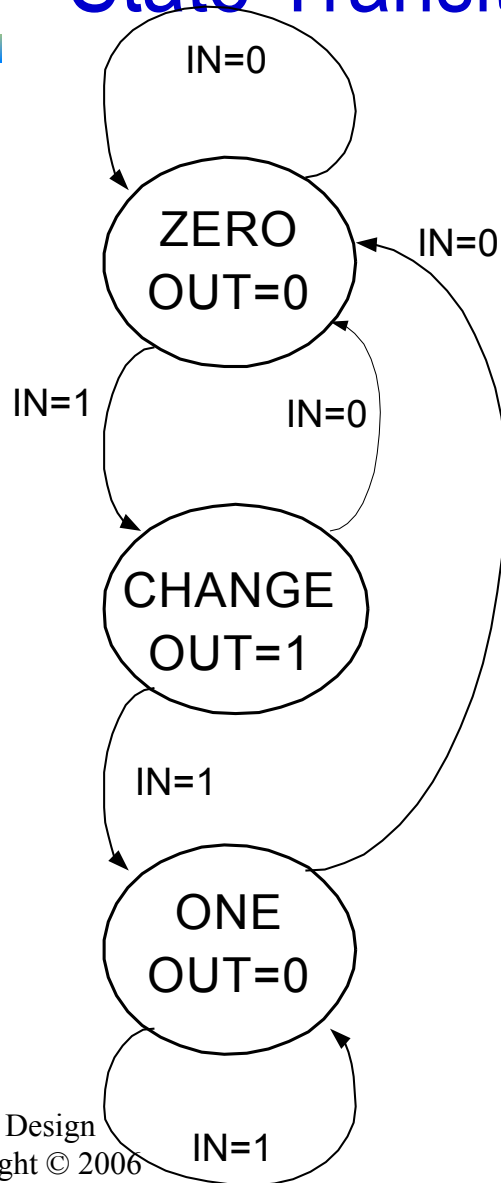
Design a circuit that asserts
its output when the input bit
stream changes
from 0 to 1.



Try two different solutions (Moore, Mealy).



State Transition Diagram - Moore

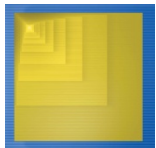
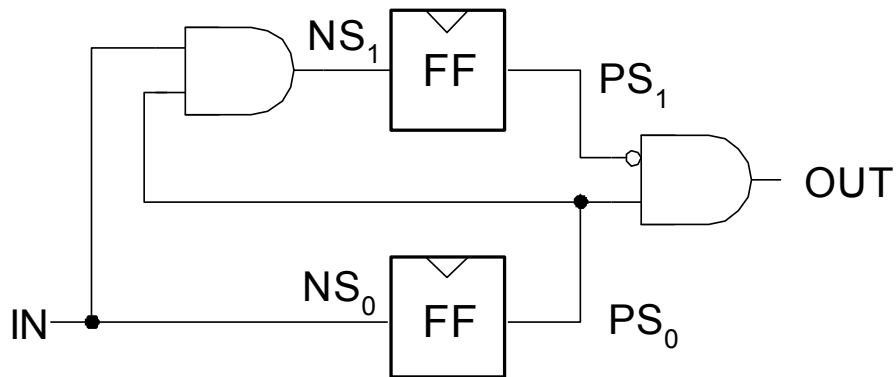


	IN	PS	NS	OUT
ZERO	0	00	00	0
	1	00	01	0
CHANGE	0	01	00	1
	1	01	11	1
ONE	0	11	00	0
	1	11	11	0



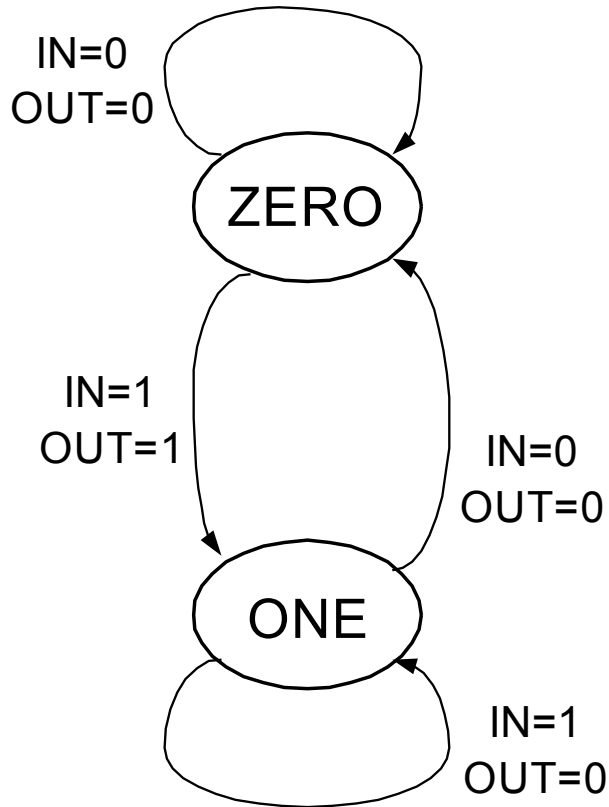
Moore FSM, circuit derivation

	IN	PS	NS	OUT
ZERO	0	00	00	0
	1	00	01	0
CHANGE	0	01	00	1
	1	01	11	1
ONE	0	11	00	0
	1	11	11	0



Mealy FSM

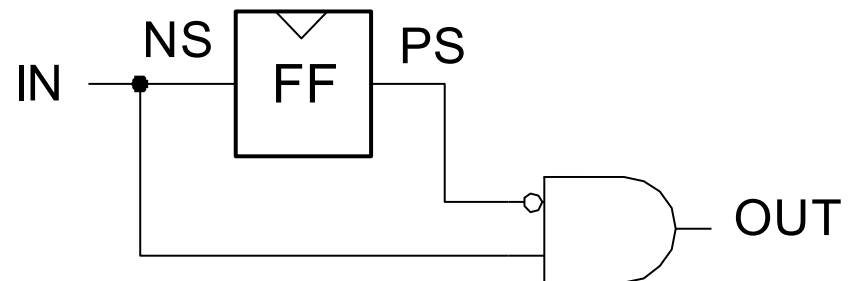
Output depends not only on PS but also on input, IN



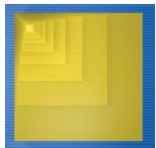
Let ZERO=0,
ONE=1

IN	PS	NS	OUT
0	0	0	0
0	1	0	0
1	0	1	1
1	1	1	0

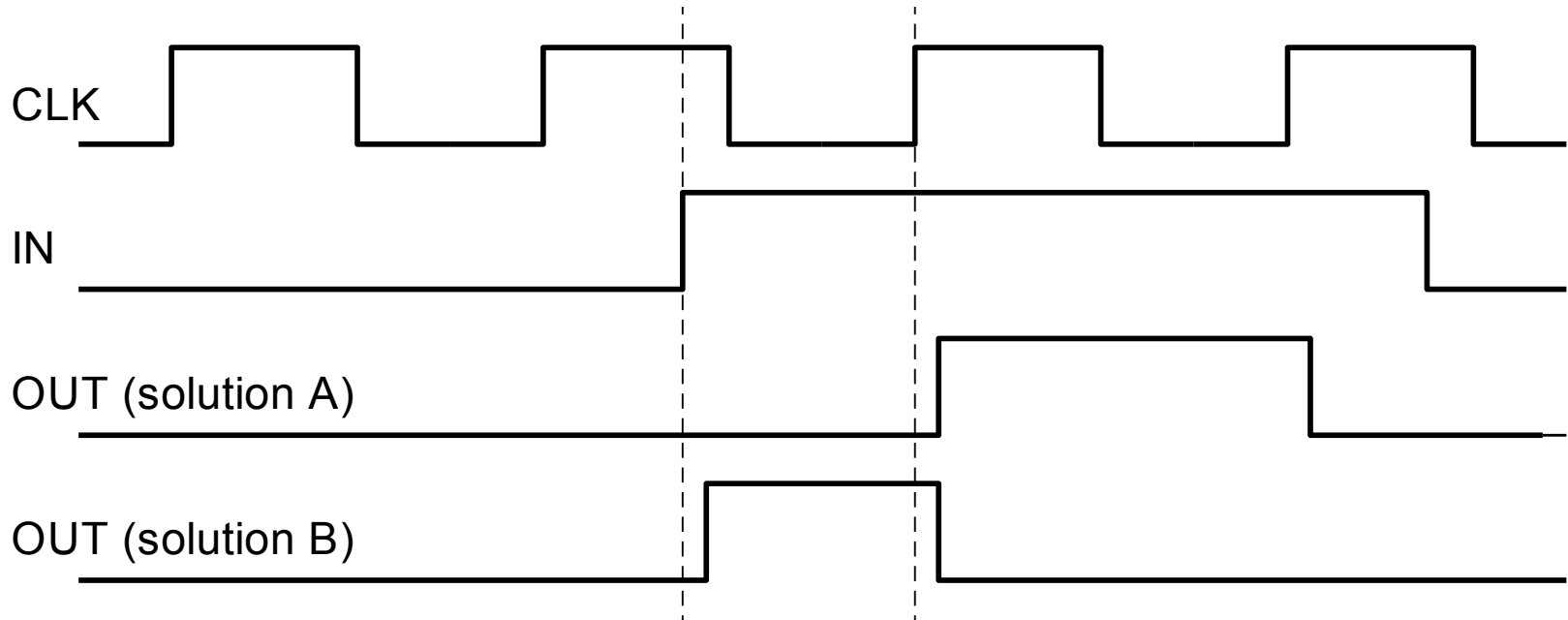
$NS = IN, OUT = IN \text{ PS}'$



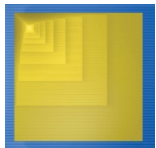
What's the *intuition* about this solution?



Edge detector timing diagrams



- Solution A (Moore): output follows the clock
- Solution B (Mealy): output changes with input rising edge and is asynchronous wrt the clock.



Chapter 3 Verilog

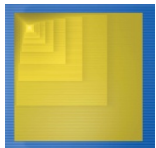
Detailed slides on Verilog slides on the course web page

```
`timescale 1 ns/1 ns

module Reg4(I, Q, Clk, Rst);

    input [3:0] I;
    output [3:0] Q;
    reg [3:0] Q;
    input Clk, Rst;

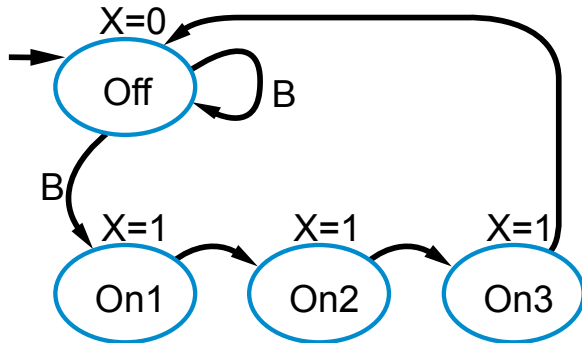
    always @(posedge Clk) begin
        if (Rst == 1 )
            Q <= 4'b0000;
        else
            Q <= I;
    end
endmodule
```



Finite-State Machines (FSMs)—Sequential Behavior

Modules with Multiple Procedures and Shared Variables

Inputs: B; Outputs: X



``timescale 1 ns/1 ns`

`module LaserTimer(B, X, Clk, Rst);`

`input B;`

`output reg X;`

`input Clk, Rst;`

`parameter S_Off = 0, S_On1 = 1,`
`S_On2 = 2, S_On3 = 3;`

`reg [1:0] State, StateNext;`

`// CombLogic`

`always @(State, B) begin`

`case (State)`

`S_Off: begin`

`X <= 0;`

`if (B == 0)`

`StateNext <= S_Off;`

`else`

`StateNext <= S_On1;`

`end`

`...`

`...`

`S_On1: begin`

`X <= 1;`

`StateNext <= S_On2;`

`end`

`S_On2: begin`

`X <= 1;`

`StateNext <= S_On3;`

`end`

`S_On3: begin`

`X <= 1;`

`StateNext <= S_Off;`

`end`

`endcase`

`end`

`// StateReg`

`always @(posedge Clk) begin`

`if (Rst == 1)`

`State <= S_Off;`

`else`

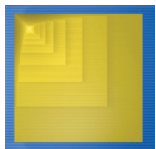
`State <= StateNext;`

`end`

`endmodule`

- Code details on Verilog slides

`vldd_ch3_LaserTimerBeh.v`



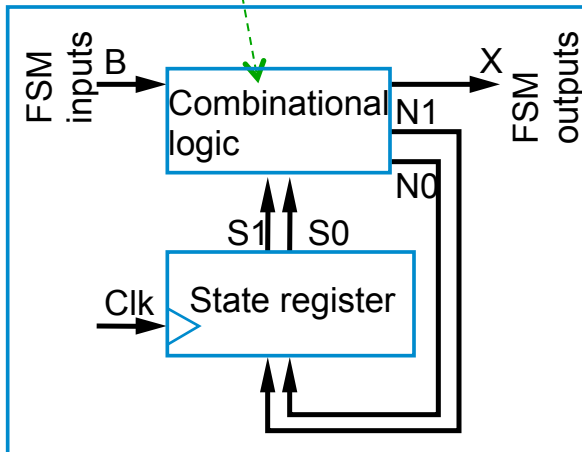
Controller Structure

- Structural description
- Test with LaserTimerTB
 - Same results

$$X = S1 + S0$$

$$N1 = S1' S0 + S1 S0'$$

$$N0 = S1' S0' B + S1 S0'$$



```

`timescale 1 ns/1 ns

module LaserTimer(B, X, Clk, Rst);

    input B;
    output reg X;
    input Clk, Rst;

    parameter S_Off = 2'b00;

    reg [1:0] State, StateNext;
    // State encodings:
    //     S_Off 00, S_On1 01, S_On2 10, S_On3 11

    // CombLogic
    always @(State, B) begin
        X <= State[1] | State[0];
        StateNext[1] <= (~State[1] & State[0])
            | (State[1] & ~State[0]);
        StateNext[0] <= (~State[1] & ~State[0] & B)
            | (State[1] & ~State[0]);
    end

    // StateReg
    always @(posedge Clk) begin
        if (Rst == 1 )
            State <= S_Off;
        else
            State <= StateNext;
        end
    endmodule
    
```

